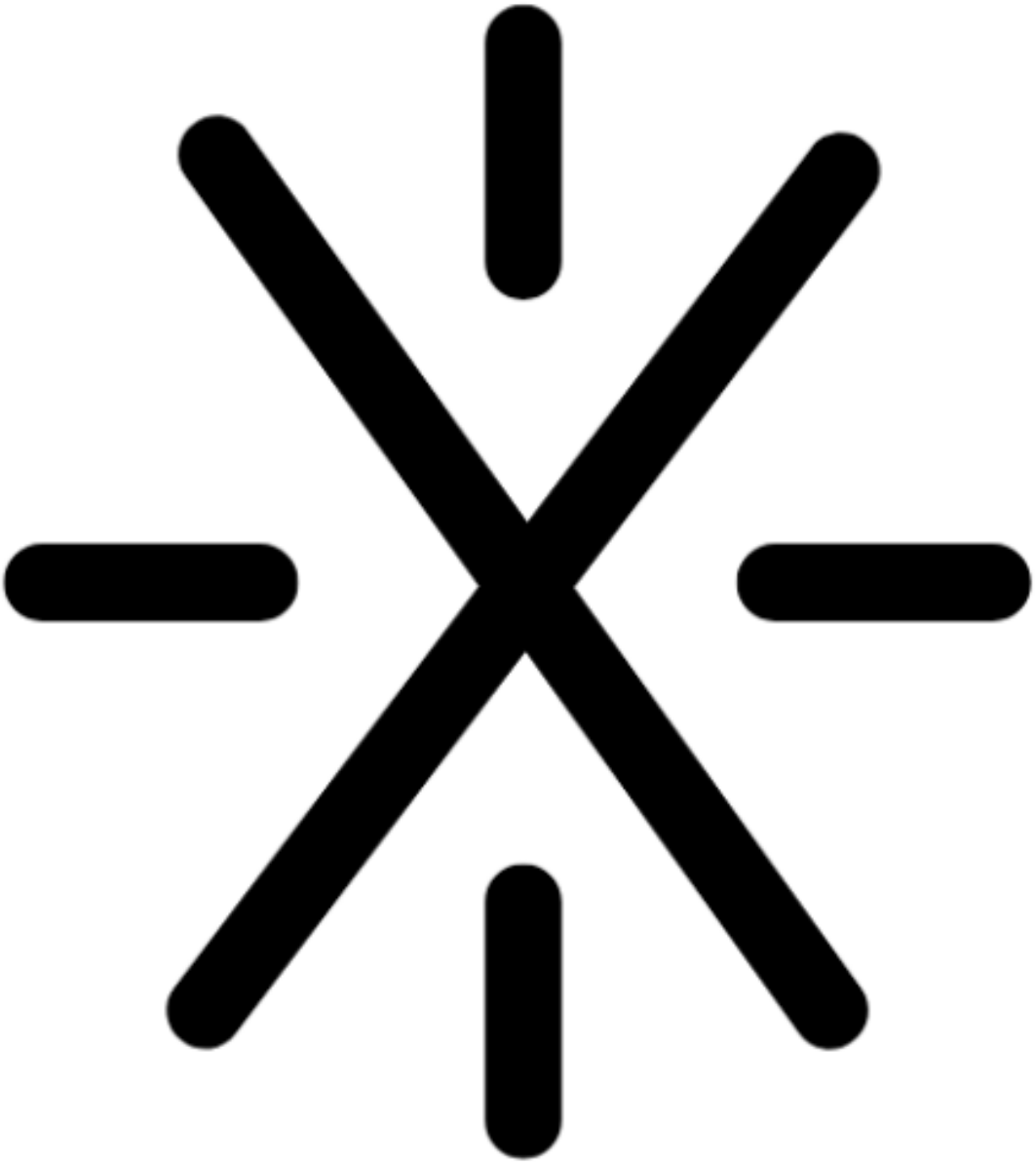

XerisCoin

Testnet SDK Reference

v1.4 | March 2026

Xeris Technologies LLC | Wyoming, USA

Seed Node: 138.197.116.81

RPC Port: 56001 | Explorer Port: 50008 | P2P Port: 4000

Table of Contents

1. Overview

2. Network Constants

3. Transaction Lifecycle

4. Instruction Reference

5. RPC Endpoints (Port 56001)

6. Explorer API (Port 50008)

7. JSON-RPC Methods (Port 50008)

8. Code Examples

9. Error Handling

10. Testnet Faucet

Appendix A: JavaScript SDK Guide

Appendix B: Test Vectors

Appendix C: Building DeFi dApps

1. Overview

XerisCoin (XRS) is a Layer 1 blockchain built entirely in Rust from scratch. It uses a triple-consensus mechanism combining Proof of Work (scrypt-based), Proof of History, and Federated Proof of Stake. The chain produces blocks every four seconds with a target throughput of 2,500 transactions per second.

This document is the complete API reference for building applications on the XerisCoin testnet. It covers every RPC endpoint, every on-chain instruction variant, and the full transaction lifecycle from construction through confirmation. All code examples use real endpoint paths and data formats that match the live testnet node.

Getting Started

The fastest way to start building on XerisCoin is with the official JavaScript SDK, available on npm. Install it, generate a keypair, and submit your first transaction in under a minute.

```
npm install xeris-sdk
```

```
const { XerisClient, XerisKeypair } = require("xeris-sdk");




async function main() {

  const client = XerisClient.testnet();

  const wallet = XerisKeypair.generate();

  console.log("Address:", wallet.publicKey);




  await client.airdrop(wallet.publicKey, 10);

  await new Promise(r => setTimeout(r, 5000));




  const balance = await client.getBalance(wallet.publicKey);

  console.log("Balance:", balance / 1_000_000_000, "XRS");
```

```
}

main();
```

The SDK handles all the complexity of bincode encoding, Solana transaction wrapping, Ed25519 signing, and RPC communication. A complete guide to every SDK method is in Appendix A. The rest of this document covers the underlying protocol details, endpoint specifications, and instruction formats for developers who want to build in other languages or need to understand the wire format.

Architecture

The testnet node exposes three network services. The RPC server on port 56001 handles transaction submission and chain state queries. The Explorer API on port 50008 provides paginated block and transaction data along with a Solana-compatible JSON-RPC interface. The P2P listener on port 4000 handles node-to-node block propagation and synchronization.

All amounts in the XerisCoin protocol are denominated in lamports, where 1 XRS equals 1,000,000,000 lamports (nine decimal places). This applies to balances, transfer amounts, fees, and staking values throughout the entire API surface.

2. Network Constants

The following constants govern the behavior of the XerisCoin testnet. These values are compiled into the node binary and are consistent across all nodes in the network.

Constant	Value	Description
Testnet Seed Node	`138.197.116.81`	Primary bootstrap node
RPC Port	`56001`	REST API for tx submission and queries
Explorer Port	`50008`	Block explorer and JSON-RPC
P2P Port	`4000`	Node-to-node block propagation
Decimals	`9`	1 XRS = 1,000,000,000 lamports
Block Time	`4 seconds`	Target slot duration
Max TPS	`~2,500`	40,000 txs per 4-second block
Transaction Fee	`0.001 XRS`	1,000,000 lamports per transaction
Fee Activation Slot	`1,030,000`	Fees enforced from this slot onward
Min Stake to Mine	`1,000 XRS`	PoS gate for block production
Unbonding Period	`151,200 slots`	Approximately 7 days at 4s per slot
Attestation Reward	`0.01 XRS`	10,000,000 lamports per valid attestation
Attestation Rate Limit	`1 per 10 blocks`	Maximum one reward per 40 seconds per validator
Blockhash Expiry	`150 slots`	Transactions expire after 150 blocks (10 min)
Total Supply Cap	`700,000,000 XRS`	Hard cap on all emissions including mining
Testnet Treasury	`2,000,000 XRS`	Initial supply allocated to treasury

3. Transaction Lifecycle

Every transaction on XerisCoin follows a five-step lifecycle from construction to on-chain confirmation. Understanding this flow is essential for building any application that writes to the chain.

Step 1: Fetch a Recent Blockhash

Before building a transaction, you need a recent blockhash from the chain. This hash serves as a replay protection mechanism. Transactions referencing a blockhash older than 150 slots (approximately ten minutes) will be rejected. Fetch the latest blockhash by sending a JSON-RPC request to the explorer port.

```
POST http://138.197.116.81:50008/

Content-Type: application/json
```

```
{"jsonrpc":"2.0","id":1,"method":"getLatestBlockhash","params":[]}
```

The response includes the blockhash as a hex string and the last valid block height. Parse the hex string and convert it to 32 bytes for use in the transaction.

Step 2: Build the Instruction Payload

XerisCoin instructions are serialized using bincode encoding. Each instruction begins with a 4-byte little-endian unsigned integer that identifies the variant type. The remaining bytes contain the variant's fields, also in bincode format. String fields are encoded as a u64 length prefix followed by the UTF-8 bytes of the string. Integer fields are encoded as little-endian u64 values.

For example, to build a NativeTransfer instruction (variant 11), you would write the variant index as four bytes, followed by the sender address string (length-prefixed), the recipient address string (length-prefixed), and the amount as a u64 in little-endian byte order.

Step 3: Wrap in a Solana Transaction

XerisCoin uses the Solana SDK transaction format for wire encoding. Create a Solana Message containing your instruction data, then wrap it in a Transaction object. The message requires the signer's public key in the account_keys array and the instruction

data in the instructions array. Set the recent_blockhash field to the value obtained in Step 1.

Step 4: Sign and Encode

Sign the transaction with the sender's Ed25519 keypair using the Solana SDK signing method. This produces a 64-byte signature that is prepended to the transaction. After signing, serialize the entire Transaction using bincode, then encode the resulting bytes as a base64 string.

Step 5: Submit

Send the base64-encoded transaction to the node's RPC endpoint. The node validates the signature, checks for replay, and inserts the transaction into the mempool. It will be included in the next block produced by the network.

```
POST http://138.197.116.81:56001/submit

Content-Type: application/json




{"tx_base64": "<base64-encoded-transaction>"}
```

A successful response returns the transaction signature, which you can use to query the transaction status through the explorer API.

```
{"status": "ok", "signature": "5UxM3..."}
```

4. Instruction Reference

The XerisInstruction enum defines all on-chain operations. Each variant is identified by a u32 index (little-endian) as the first four bytes of the instruction data. The remaining bytes are the bincode-serialized fields for that variant. The table below lists every variant, its index, fields, and purpose.

Index	Variant	Fields	Description
0	TokenMint	token_id, to, amount	Mint new tokens (mint authority only)
1	TokenTransfer	token_id, from, to, amount	Transfer tokens between accounts
2	TokenBurn	token_id, from, amount	Burn tokens, reducing current supply
3	TokenCreate	token_id, name, symbol, decimals, max_supply, mint_authority	Register a new token on-chain
4	ContractCall	contract_id, method, args	Call a method on a deployed contract
5	ContractDeploy	contract_id, contract_type_str, params_json	Deploy a new smart contract
6	TokenCreateRWA	token_id, name, symbol, decimals, max_supply, mint_authority, asset_type, legal_doc_hash, legal_doc_uri, jurisdiction, transfer_restricted, accredited_only, valuation	Create a real-world asset backed token
7	RWAUpdateStatus	token_id, new_status, new_valuation, new_legal_doc_hash, new_legal_doc_uri	Update metadata on an existing RWA token
8	RWATransfer	token_id, from, to, amount	Transfer RWA tokens with compliance checks
9	Stake	pubkey, amount	Stake XRS for PoS mining eligibility
10	Unstake	pubkey, amount	Begin unstaking with unbonding period
11	NativeTransfer	from, to, amount	Transfer native XRS between accounts

12	ValidatorAttestation	validator, block_slot, block_hash_prefix	Submit block verification proof for reward
13	WrapXrs	amount	Convert native XRS to xrs_native token
14	UnwrapXrs	amount	Convert xrs_native token back to native XRS

Field Encoding Details

All string fields in bincode are encoded as a u64 length prefix (8 bytes, little-endian) followed by the raw UTF-8 bytes. For example, the string "abc" is encoded as the bytes 03 00 00 00 00 00 00 00 followed by 61 62 63. All integer fields (u64) are 8 bytes in little-endian order. Boolean fields are a single byte where 0x00 is false and 0x01 is true. The Vec<u8> type used in block_hash_prefix and args is encoded identically to strings, as a u64 length prefix followed by the raw bytes.

Contract Types

When deploying a contract with variant 5 (ContractDeploy), the contract_type_str field accepts the following values. Each type has its own state model and execution methods.

Type String	Description
timelock	Time-locked token release with configurable schedule
escrow	Two-party escrow with conditional release
swap	Automated market maker liquidity pool (0.777% fee)
vesting	Token vesting with cliff and linear release
multisig	Multi-signature wallet requiring M-of-N approvals
rwa	Real-world asset management contract
launchpad	Pump.fun-style bonding curve token launcher

5. RPC Endpoints (Port 56001)

The RPC server is the primary interface for submitting transactions and querying chain state. All endpoints return JSON responses. Write endpoints (POST) are rate-limited to 30 requests per minute per IP address.

Transaction Submission

Method	Path	Description
POST	`/submit`	Submit a signed, base64-encoded transaction
POST	`/stake`	Submit a signed staking transaction
POST	`/unstake`	Submit a signed unstaking transaction

POST /submit

This is the primary endpoint for all transaction types. The request body must contain a single field called tx_base64, which holds the base64-encoded bincode serialization of a signed Solana Transaction.

```
Request:  {"tx_base64": "<base64-encoded-transaction>"}



Success:  {"status": "ok", "signature": "5UxM3..."}



For attestation txs that pass validation:

{"status":"ok", "signature":"...", "attestation_accepted":true,

 "reward":10000000, "reward_xrs":0.01}



Rate-limited attestation:
```

```
{"error":"Attestation rate-limited", "status":"rate_limited",

 "next_eligible_slot":1133760, "seconds_remaining":24}
```

Chain State Queries

Method	Path	Description
GET	`/health`	Node health check
GET	`/blocks`	Last 50 blocks (full Block objects)
GET	`/stake/{address}`	Stake info for an address
GET	`/network/economics`	Chain economics dashboard

GET /network/economics

Returns a comprehensive snapshot of the chain's economic state including total supply mined, fees collected, active validators, current difficulty, and emission schedule data. This is useful for building dashboards or analytics tools.

Token Operations

Method	Path	Description
GET	`/token/balance/{addr}/{token_id}`	Balance for a specific token
GET	`/token/accounts/{address}`	All token balances for a wallet (including native XRS)
GET	`/tokens`	List all registered tokens
POST	`/token/mint`	Submit signed TokenMint tx via /submit instead
POST	`/token/transfer`	Submit signed TokenTransfer tx via /submit instead
POST	`/token/burn`	Submit signed TokenBurn tx via /submit instead
POST	`/token/create`	Submit signed TokenCreate tx via /submit instead

The POST token endpoints return guidance to use /submit with a signed transaction instead. All token mutations must go through the mempool as signed XerisInstruction transactions. The GET endpoints are fully functional for reading token state.

Smart Contracts

Method	Path	Description
GET	`/contracts`	List all deployed contracts with state
GET	`/contract/{id}`	Get a specific contract's full state
GET	`/contract/{id}/quote?...`	Preview a swap or contract operation
POST	`/contract/deploy`	Submit signed ContractDeploy tx via /submit instead
POST	`/contract/call`	Submit signed ContractCall tx via /submit instead

Launchpad (Bonding Curves)

Method	Path	Description
GET	`/launchpads`	List all active bonding curves with price data
GET	`/launchpad/{id}/quote?xrs_amount=N`	Preview a buy before executing

GET /launchpad/{id}/quote

Returns a preview of what a buy operation would yield without executing it. The response includes the number of tokens the buyer would receive, the creator fee (1%), the Xeris protocol fee (0.777%), the effective price per token, and the price impact percentage. Use the tokens_out value multiplied by 0.95 as the min_tokens_out parameter for 5% slippage tolerance when submitting the actual buy transaction.

Alexandria Protocol (Real-World Assets)

Method	Path	Description
POST	`/rwa/create`	Submit signed TokenCreateRWA tx via /submit instead
POST	`/rwa/update_status`	Submit signed RWAUpdateStatus tx via /submit instead

Testnet Faucet

Method	Path	Description
GET	`/airdrop/{address}/{amount}`	Request testnet XRS (amount in lamports)

The airdrop endpoint is rate-limited to one request per ten seconds per IP address. The amount parameter is specified in lamports. To request 100 XRS, pass 100000000000 as the amount. This endpoint is disabled on mainnet.

6. Explorer API (Port 50008)

The Explorer API provides paginated access to blocks, transactions, accounts, and validators. All paginated endpoints accept optional page and page_size query parameters with a maximum page size of 100. Responses include a pagination object with total count, current page, page size, and total pages.

Method	Path	Description
GET	`/v2/stats`	Network overview (height, supply, difficulty, TPS)
GET	`/v2/blocks?page=1&page_size=20`	Paginated block list
GET	`/v2/block/slot/{slot}`	Block detail by slot number
GET	`/v2/block/hash/{hash}`	Block detail by hash (hex string)
GET	`/v2/transactions?page=1&page_size=20`	Paginated recent transactions
GET	`/v2/tx/{signature}`	Full transaction detail by signature
GET	`/v2/account/{address}`	Account info (balance, stake, blocks proposed)
GET	`/v2/account/{address}/transactions`	Transaction history filtered by address
GET	`/v2/validators`	All validators with stake and proposed block counts
GET	`/v2/tokens`	All registered tokens with metadata
GET	`/v2/search?q={query}`	Universal search (address, signature, slot, hash)

Transaction Detail Response

The /v2/tx/{signature} endpoint returns fully decoded transaction data. Every instruction variant is parsed from the raw bincode payload, so the response always includes populated from, to, amount, and tx_type fields. The tx_type field contains a human-readable label such as native_transfer, token_transfer, stake, attestation, contract_call:buy_tokens, or any of the other instruction types. For transactions with

additional context such as token operations or contract calls, a details object is included with fields like token_id, contract_id, and method.


```
{
  "success": true,
  "data": {
    "signature": "5UxM3...",
    "block_slot": 1133754,
    "block_hash": "0a1b2c...",
    "poh_timestamp": 1709571234000,
    "from": "8evPjj...",
    "to": "3xKm9P...",
    "amount": 5000000000,
    "amount_xrs": 5.0,
    "tx_type": "native_transfer",
    "details": null,
    "status": "confirmed",
    "confirmations": 42
  }
}
```


Transaction Type Values

The tx_type field in transaction responses maps to the following values based on the decoded instruction variant.

tx_type	Instruction
native_transfer	NativeTransfer (variant 11)

token_transfer	TokenTransfer (variant 1)
token_mint	TokenMint (variant 0)
token_burn	TokenBurn (variant 2)
token_create	TokenCreate (variant 3)
rwa_create	TokenCreateRWA (variant 6)
rwa_update	RWAUpdateStatus (variant 7)
rwa_transfer	RWATransfer (variant 8)
stake	Stake (variant 9)
unstake	Unstake (variant 10)
attestation	ValidatorAttestation (variant 12)
contract_deploy	ContractDeploy (variant 5)
contract_call:METHOD	ContractCall (variant 4, METHOD is the method name)
wrap_xrs	WrapXrs (variant 13)
unwrap_xrs	UnwrapXrs (variant 14)
legacy_transfer	Legacy SystemInstruction::Transfer (pre-v1.2 blocks)

7. JSON-RPC Methods (Port 50008)

The explorer port also serves a Solana-compatible JSON-RPC interface at the root path. All requests use HTTP POST with a standard JSON-RPC 2.0 envelope. This interface is useful for wallet integrations and tools that already support the Solana RPC format.

Method	Parameters	Returns
getLatestBlockhash	none	Blockhash (hex) and lastValidBlockHeight
getBalance	[address]	Balance in lamports
getAccountInfo	[address]	Balance, stake, validator status, token balances
getSlot	none	Current chain height (slot number)
getBlockHeight	none	Current chain height
getBlock	[slot]	Full block with decoded transactions
getTransaction	[signature]	Full transaction with from, to, amount, tx_type
getSignaturesForAddress	[address, {limit}]	Recent transactions involving this address
getHealth	none	Returns "ok"
getVersion	none	Node software version

Example: getBalance

```
POST http://138.197.116.81:50008/
```

```
{"jsonrpc":"2.0","id":1,"method":"getBalance",

 "params":["8evPjjozSHNcoGRcv7zzxwan9sf3ubJ8q9CFzms6AK97"]}
```

```
Response:
```

```
{"jsonrpc":"2.0","id":1,

 "result":{"context":{"slot":1133754},"value":1500000000000}}
```


Example: getTransaction

```
{"jsonrpc":"2.0","id":1,"method":"getTransaction",

 "params":["5UxM3..."]}
```

Response includes decoded from, to, amount, tx_type, and details.

8. Code Examples

The following examples demonstrate common operations against the XerisCoin testnet. All examples use standard HTTP requests with JSON payloads. The transaction-building examples show the conceptual flow, which can be implemented in any language that supports Ed25519 signing and bincode serialization.

Check Balance (JavaScript)

```
async function getBalance(address) {
  const resp = await fetch("http://138.197.116.81:50008/", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({
      jsonrpc: "2.0", id: 1,
      method: "getBalance",
      params: [address]
    })
  });
  const data = await resp.json();
  const lamports = data.result.value;
  return lamports / 1_000_000_000; // XRS
}
```

Get All Token Balances (JavaScript)

```
async function getTokenAccounts(address) {
  const resp = await fetch(
    `http://138.197.116.81:56001/token/accounts/${address}`
```

```
  );

  const data = await resp.json();

  // data.native_xrs = native balance in lamports

  // data.token_accounts = [{token_id, symbol, balance, balance_display}]

  return data;

}
```

Request Testnet Airdrop (curl)

```
# Request 100 XRS to your address

curl http://138.197.116.81:56001/airdrop/YOUR_ADDRESS/100000000000
```

Query Launchpad Curves (JavaScript)

```
async function getActiveLaunchpads() {

  const resp = await fetch("http://138.197.116.81:56001/launchpads");

  return resp.json();

  // Returns array of {contract_id, token_id, name, symbol,

  //   current_price, market_cap, progress_pct, ...}

}



async function getQuote(contractId, xrsAmount) {

  const resp = await fetch(

    `http://138.197.116.81:56001/launchpad/${contractId}/quote?xrs_amount=${xrsAmount}
`

  );

  return resp.json();
```

```
  // Returns {tokens_out, creator_fee, xeris_fee, effective_price,
  //   price_after, price_impact_pct}
}
```

Building a NativeTransfer (Pseudocode)

This pseudocode shows the complete flow for constructing, signing, and submitting a native XRS transfer. Adapt this pattern to your language of choice using an Ed25519 library and a bincode serializer.

```
// 1. Get blockhash
blockhash = rpc("getLatestBlockhash").value.blockhash
blockhash_bytes = hex_decode(blockhash)



// 2. Build instruction data (variant 11 = NativeTransfer)
ix_data = []
ix_data.append(u32_le(11))           // variant index
ix_data.append(bincode_string(from))  // sender address
ix_data.append(bincode_string(to))    // recipient address
ix_data.append(u64_le(amount))        // amount in lamports



// 3. Create Solana Message
message = SolanaMessage {
  account_keys: [sender_pubkey],
  recent_blockhash: blockhash_bytes,
  instructions: [{program_id_index: 0, accounts: [], data: ix_data}]
}
```

```
// 4. Sign
signature = ed25519_sign(keypair, message.serialize())


// 5. Build Transaction and submit
tx = SolanaTransaction { signatures: [signature], message: message }
tx_bytes = bincode_serialize(tx)
tx_base64 = base64_encode(tx_bytes)


response = http_post("/submit", {tx_base64: tx_base64})
// response: {"status":"ok", "signature":"5UxM3..."}
```

Building a Stake Transaction (Pseudocode)

```
// Variant 9 = Stake
ix_data = []
ix_data.append(u32_le(9))              // variant index
ix_data.append(bincode_string(pubkey))  // validator pubkey
ix_data.append(u64_le(stake_amount))    // amount in lamports


// Follow steps 3-5 from NativeTransfer example above
```

Submitting an Attestation (Pseudocode)

```
// Variant 12 = ValidatorAttestation
ix_data = []
```

```
ix_data.append(u32_le(12))                    // variant index

ix_data.append(bincode_string(my_pubkey))      // validator address

ix_data.append(u64_le(block_slot))             // slot of verified block

ix_data.append(bincode_vec(block_hash_bytes))  // full 32-byte block hash



// Submit via /submit. Check response:

// attestation_accepted: true  -> reward will be paid

// status: rate_limited        -> wait seconds_remaining
```

9. Error Handling

All RPC and Explorer endpoints return JSON responses. Errors are indicated by an error field in the response body. There are no HTTP status code errors. A 200 response with an error field means the request was received but could not be fulfilled.

Error Message	Cause	Resolution
Invalid base64 encoding	tx_base64 is not valid base64	Check your base64 encoding step
Failed to deserialize transaction	Bincode payload is malformed	Verify bincode serialization matches Solana Transaction format
Invalid transaction signature	Ed25519 signature does not match the message	Ensure you are signing the correct message bytes with the correct keypair
Transaction already processed	This signature is already on-chain	This transaction was already confirmed. No action needed.
Transaction already in mempool	This signature is already queued	Wait for the current submission to be included in a block
Rate limited	Too many write RPCs from this IP	Wait 60 seconds before retrying. Limit is 30 writes per minute.
Attestation rate-limited	Already attested within the last 10 blocks	Wait for seconds_remaining from the response before next attestation
Attestation rejected: block too old	Attested block is older than 200 slots	Attest to a more recent block
Attestation rejected: block hash mismatch	Hash prefix does not match the block	Ensure you are sending the full 32-byte hash of the correct block
Transaction not found	Signature not in recent blocks	The transaction may be in a block older than the node's recent block window

10. Testnet Faucet

The testnet faucet provides free XRS tokens for development and testing. It is accessible through a simple GET request and does not require authentication or a signed transaction. The faucet is funded by the testnet treasury and is rate-limited to prevent abuse.

Endpoint

```
GET http://138.197.116.81:56001/airdrop/{address}/{amount_lamports}
```

Parameters

The address parameter is the recipient's base58-encoded public key. The amount_lamports parameter is the number of lamports to send, where 1,000,000,000 lamports equals 1 XRS. The maximum airdrop amount per request is determined by the treasury's available balance.

Rate Limiting

The faucet is rate-limited to one request per ten seconds per IP address. If you exceed this limit, the endpoint will return an error message. Wait ten seconds and try again.

Example

```
# Request 10 XRS

curl http://138.197.116.81:56001/airdrop/YOUR_PUBKEY/10000000000




# Response:

{"status":"ok","signature":"4kPw9...","amount_xrs":10.0}
```

Appendix A: JavaScript SDK Guide

The xeris-sdk package is the official JavaScript client library for the XerisCoin blockchain. It is available on npm and handles all the complexity of bincode instruction encoding, Solana transaction wrapping, Ed25519 signing, and RPC communication. This guide walks through every step from installation to building a complete application.

Installation

The SDK requires Node.js version 16 or higher. Create a new project directory, initialize it, and install the SDK. The two dependencies (@solana/web3.js and bs58) are pulled in automatically.

```
mkdir my-xeris-app && cd my-xeris-app

npm init -y

npm install xeris-sdk
```

Once installed, import the two core classes at the top of your file. XerisClient handles all network communication and XerisKeypair manages your signing keys.

```
const { XerisClient, XerisKeypair } = require("xeris-sdk");
```

TypeScript users get full type definitions out of the box. The package includes an index.d.ts file so all methods have autocomplete and type checking in your editor.

```
import { XerisClient, XerisKeypair } from "xeris-sdk";
```

Connecting to the Network

The XerisClient constructor takes a host URL and optional port configuration. For testnet development, use the static testnet() method which points at the public seed node automatically.

```
// Connect to the public testnet

const client = XerisClient.testnet();




// Or connect to your own node

const client = new XerisClient("http://your-node-ip", {
```

```
  rpcPort: 56001,

  explorerPort: 50008

});
```

The client communicates with two ports on the node. Port 56001 is the RPC server that handles transaction submission and chain state queries. Port 50008 is the Explorer API that provides paginated data and a Solana-compatible JSON-RPC interface. The SDK routes requests to the correct port automatically based on which method you call.

You can verify the connection by calling getHealth(). A healthy node returns an object with status set to ok.

```
const health = await client.getHealth();

console.log(health); // { status: "ok" }
```

Managing Keypairs

Every transaction on XerisCoin must be signed with an Ed25519 keypair. The XerisKeypair class provides methods for generating, saving, and loading keypairs. The file format is a JSON array of 64 bytes, which is the same format used by the Rust CLI wallet, so keypairs are fully interchangeable between the SDK and the command-line tools.

```
// Generate a brand new keypair

const alice = XerisKeypair.generate();

console.log("Address:", alice.publicKey);




// Save it to a file for later use

alice.saveToFile("alice-keypair.json");




// Load it back from the file

const loaded = XerisKeypair.fromJsonFile("alice-keypair.json");
```

```
// Load from raw secret key bytes

const fromBytes = XerisKeypair.fromSecretKey(secretKeyArray);
```

Keep your keypair files safe. Anyone with access to the JSON file can sign transactions as that address. Never commit keypair files to source control or share them publicly.

Getting Testnet XRS

The testnet faucet provides free XRS for development. The airdrop method takes an address and an amount in whole XRS. The maximum per request is 10 XRS, and there is a cooldown of 10 seconds between requests from the same IP address. The airdrop is processed as a transaction from the treasury, so it takes one block (approximately four seconds) to appear in your balance.

```
// Request 10 XRS

const result = await client.airdrop(alice.publicKey, 10);

console.log(result);

// { status: "Airdrop Transaction Queued",

//   recipient: "9QqCS...", amount_xrs: 10 }




// Wait for the next block

await new Promise(r => setTimeout(r, 5000));




// Verify it arrived

const balance = await client.getBalance(alice.publicKey);

console.log(balance / 1_000_000_000, "XRS");  // 10 XRS
```

Checking Balances

The SDK provides several ways to check balances depending on what information you need. The getBalance method returns native XRS in lamports through the JSON-RPC interface. The getTokenAccounts method returns all token balances for an address along

with the native XRS balance. The getAccountInfo method from the Explorer API returns balance, stake status, validator status, and the number of blocks proposed.

```
// Native XRS balance (in lamports)

const lamports = await client.getBalance(alice.publicKey);

const xrs = lamports / 1_000_000_000;




// All token balances + native XRS

const accounts = await client.getTokenAccounts(alice.publicKey);

// accounts.native_xrs = lamports

// accounts.token_accounts = [

//   { token_id: "xUSDC", symbol: "USDC", balance: 5000000000,

//     balance_display: 5.0 }

// ]




// Full account info (balance, stake, validator status)

const info = await client.getAccountInfo(alice.publicKey);
```

Transferring XRS

The transferXrs method handles the complete transaction lifecycle in a single call. It fetches a recent blockhash, builds a NativeTransfer instruction with the correct bincode encoding, wraps it in a Solana transaction, signs it with the provided keypair, serializes it to base64, and submits it to the node. The method returns the transaction signature which you can use to track its status.

```
const bob = XerisKeypair.generate();




// Send 2 XRS to Bob
```

```
const tx = await client.transferXrs(alice, bob.publicKey, 2.0);

console.log("Signature:", tx.signature);



// Wait for block inclusion (one block = ~4 seconds)

await new Promise(r => setTimeout(r, 5000));



// Verify the transaction on-chain

const detail = await client.getTransaction(tx.signature);

console.log(detail.data.tx_type);    // "native_transfer"

console.log(detail.data.amount_xrs); // 2

console.log(detail.data.status);     // "confirmed"
```

A transaction fee of 0.001 XRS is deducted from the sender's balance for every confirmed transaction. Transactions referencing a blockhash older than 150 blocks (approximately ten minutes) will be rejected, so submit promptly after building.

Staking XRS

Staking is required for Proof of Stake mining eligibility. The minimum stake is 1,000 XRS. Once staked, the validator must also propose blocks to remain in the active set for leader selection. Unstaking initiates a 7-day unbonding period during which the funds cannot be transferred.

```
// Stake 1000 XRS

await client.stakeXrs(alice, 1000);



// Check stake status

const stakeInfo = await client.getStakeInfo(alice.publicKey);

console.log(stakeInfo);
```

```
// Begin unstaking (starts 7-day unbonding)

await client.unstakeXrs(alice, 500);
```

Working with Tokens

XerisCoin supports custom tokens with configurable supply, decimals, and mint authority. The full token lifecycle is creating the token, minting supply, transferring between accounts, and optionally burning. All token operations are on-chain transactions that flow through the mempool and are included in blocks.

```
// Step 1: Create a token

await client.createToken(

  alice,        // keypair (becomes mint authority)

  "mytoken",    // unique token ID

  "My Token",   // display name

  "MTK",        // ticker symbol

  9,            // decimals (9 = same as XRS)

  1000000       // max supply in whole tokens

);



// Step 2: Mint tokens to yourself

await client.mintTokens(alice, "mytoken", alice.publicKey, 10000, 9);



// Step 3: Transfer tokens to another address

await client.transferToken(alice, "mytoken", bob.publicKey, 500, 9);
```

```
// Step 4: Burn tokens (optional)

await client.burnTokens(alice, "mytoken", 100, 9);



// Check token balance

const bal = await client.getTokenBalance(bob.publicKey, "mytoken");

console.log(bal);



// List all tokens on the chain

const tokens = await client.getTokenList();
```

Smart Contracts

XerisCoin supports seven built-in contract types: swap (AMM liquidity pool), escrow, timelock, vesting, multisig, rwa (real-world assets), and launchpad (bonding curve token launcher). Contracts are deployed with an ID, a type, and initialization parameters. Once deployed, you interact with them by calling named methods.

Deploying a Liquidity Pool

A swap contract creates an automated market maker pool between two tokens. The fee_bps parameter is the swap fee in basis points, where 77 equals 0.77%. Both tokens must exist and the deployer must have sufficient balances of each.

```
await client.deployContract(alice, "pool_mtk_xrs", "swap", {

  token_a: "mytoken",

  token_b: "xrs_native",

  amount_a: 100000000000000,  // 100,000 MTK (9 decimals)

  amount_b: 10000000000000,   // 10,000 XRS (9 decimals)

  fee_bps: 77

});
```

```
// Query the pool state

const pool = await client.getContract("pool_mtk_xrs");

console.log(pool);
```

Calling Contract Methods

Use callContract to execute any method on a deployed contract. The method name and arguments vary by contract type. For swap contracts, common methods include add_liquidity, remove_liquidity, and swap.

```
// Swap tokens through the pool

await client.callContract(alice, "pool_mtk_xrs", "swap", {

  token_in: "mytoken",

  amount_in: 1000000000000,    // 1,000 MTK

  min_amount_out: 90000000000  // minimum 90 XRS (slippage protection)

});
```

Launchpad (Bonding Curves)

The launchpad system uses bonding curves for token price discovery. Tokens start at a low price and increase as more are purchased. Each launchpad charges a 1% creator fee and a 0.777% Xeris protocol fee. Before buying, always fetch a quote to see what you would receive and the price impact.

```
// List all active launchpads

const pads = await client.getLaunchpads();

console.log(pads);



// Get a quote before buying (1 XRS worth)

const quote = await client.getLaunchpadQuote("launch_xyz", 1000000000);
```

```
console.log("Tokens out:", quote.tokens_out);

console.log("Price impact:", quote.price_impact_pct, "%");



// Execute the buy with 5% slippage tolerance

const minOut = Math.floor(quote.tokens_out * 0.95);

await client.callContract(alice, "launch_xyz", "buy_tokens", {

  xrs_amount: 1000000000,

  min_tokens_out: minOut

});
```

Wrapping XRS for DEX Trading

The on-chain DEX operates on token balances, not native XRS balances. To trade native XRS in a liquidity pool, you first wrap it into the xrs_native token. This moves the XRS from your native balance to your token balance at a 1:1 ratio. Unwrapping converts it back.

```
// Wrap 10 XRS into xrs_native token

await client.wrapXrs(alice, 10);



// Now you can trade it in pools

// ... swap, add liquidity, etc.



// Unwrap back to native XRS

await client.unwrapXrs(alice, 10);
```

Light Client Attestation

Light client attestation is a way to earn XRS rewards by verifying blocks. Your client fetches a recent block, confirms the hash chain is valid, and submits an attestation transaction. The reward is 0.01 XRS per valid attestation, rate-limited to one reward per 10 blocks (approximately every 40 seconds) per validator. The node pre-checks the rate limit at submission time, so the response tells you immediately whether the reward was accepted or when your next eligible slot is.

```
// Fetch the most recent block

const blocks = await client.getRecentBlocks();

const block = blocks[0];

const hashBytes = Buffer.from(block.hash, "hex");




// Submit the attestation

const resp = await client.submitAttestation(alice, block.slot, hashBytes);




// Check the result

if (resp.attestation_accepted) {

  console.log("Earned 0.01 XRS");

} else if (resp.status === "rate_limited") {

  console.log("Next reward in", resp.seconds_remaining, "seconds");

}
```

Querying Chain Data

The SDK provides read-only methods for every query endpoint on both the RPC and Explorer APIs. These do not require a keypair and return parsed JSON responses. All paginated endpoints accept optional page and pageSize parameters.

Blocks and Transactions

```
// Network overview
```

```
const stats = await client.getStats();


// Paginated block list
const blocks = await client.getBlocks(1, 20);


// Single block by slot
const block = await client.getBlockBySlot(1135100);


// Transaction detail by signature
const tx = await client.getTransaction("5FAha...");


// Transaction history for an address
const history = await client.getAccountTransactions(alice.publicKey);
```

Network and Validators

```
// Economics dashboard (supply, fees, difficulty)
const econ = await client.getNetworkEconomics();


// All validators with stake amounts
const validators = await client.getValidators();


// Current slot number
const slot = await client.getSlot();
```

Universal Search

The search method accepts any string and automatically detects whether it is an address, a transaction signature, a slot number, or a block hash. It returns the matching result with its type.

```
const result = await client.search("5FAhaNi6nk...");

console.log(result);  // { type: "transaction", data: { ... } }
```

Advanced: Custom Instructions

For operations not covered by the high-level methods, or for building custom transaction pipelines, you can use the low-level Instructions namespace to build raw instruction bytes and submit them with sendInstruction. You can also use the encoding primitives directly (encodeU32, encodeU64, encodeBincodeString) to construct entirely custom payloads.

```
const { Instructions, encodeU32, encodeU64, encodeBincodeString } = require("xeris-sdk");



// Use a pre-built instruction

const ixData = Instructions.nativeTransfer(from, to, lamports);

const result = await client.sendInstruction(keypair, ixData);



// Or build completely custom bytes

const customIx = Buffer.concat([

  encodeU32(11),                      // variant index

  encodeBincodeString(fromAddress),    // bincode string

  encodeBincodeString(toAddress),

  encodeU64(amountInLamports),         // u64 little-endian

]);
```

```
const result = await client.sendInstruction(keypair, customIx);
```

Error Handling

All transaction methods throw an Error if the node returns an error response. Query methods throw on HTTP errors. Wrap your calls in try/catch to handle failures gracefully.

```
try {
  const tx = await client.transferXrs(alice, bob.publicKey, 5.0);
  console.log("Success:", tx.signature);
} catch (err) {
  console.error("Failed:", err.message);
  // Common errors:
  // "Invalid transaction signature"
  // "Transaction already processed"
  // "Rate limited"
}
```

Complete Working Example

The following script demonstrates the full lifecycle of interacting with the XerisCoin testnet through the SDK. Copy it into a file, install the SDK, and run it with Node.js. Every step waits for block confirmation before proceeding to the next.

```
const { XerisClient, XerisKeypair } = require("xeris-sdk");



async function main() {
  const client = XerisClient.testnet();
  const alice = XerisKeypair.generate();
  const bob = XerisKeypair.generate();
```

```
console.log("Alice:", alice.publicKey);
console.log("Bob:", bob.publicKey);


// Fund Alice
await client.airdrop(alice.publicKey, 10);
await new Promise(r => setTimeout(r, 5000));
const bal = await client.getBalance(alice.publicKey);
console.log("Alice balance:", bal / 1_000_000_000, "XRS");


// Send 2 XRS to Bob
const tx = await client.transferXrs(alice, bob.publicKey, 2.0);
console.log("Transfer sig:", tx.signature);
await new Promise(r => setTimeout(r, 5000));


// Verify on-chain
const detail = await client.getTransaction(tx.signature);
console.log("Type:", detail.data.tx_type);
console.log("Amount:", detail.data.amount_xrs, "XRS");
console.log("Status:", detail.data.status);


// Check Bob received it
const bobBal = await client.getBalance(bob.publicKey);
```

```
  console.log("Bob balance:", bobBal / 1_000_000_000, "XRS");

}



main().catch(console.error);
```

Available Methods Reference

The following table lists every method available on XerisClient, organized by category. Transaction methods require a XerisKeypair as the first argument. Query methods require no keypair and can be called freely.

Method	Type	Description
`transferXrs(kp, to, xrs)`	Write	Transfer native XRS
`stakeXrs(kp, xrs)`	Write	Stake XRS for mining
`unstakeXrs(kp, xrs)`	Write	Begin unstaking
`createToken(kp, id, name, sym, d, s)`	Write	Register a new token
`mintTokens(kp, id, to, amt, d)`	Write	Mint tokens
`transferToken(kp, id, to, amt, d)`	Write	Transfer tokens
`burnTokens(kp, id, amt, d)`	Write	Burn tokens
`deployContract(kp, id, type, params)`	Write	Deploy a contract
`callContract(kp, id, method, args)`	Write	Call a contract method
`wrapXrs(kp, xrs)`	Write	Wrap XRS for DEX
`unwrapXrs(kp, xrs)`	Write	Unwrap back to native
`submitAttestation(kp, slot, hash)`	Write	Attest a block for reward
`airdrop(addr, xrs)`	Read	Request testnet faucet
`getBalance(addr)`	Read	Native XRS in lamports
`getTokenAccounts(addr)`	Read	All token balances
`getAccountInfo(addr)`	Read	Full account info
`getStakeInfo(addr)`	Read	Stake and validator status
`getTransaction(sig)`	Read	Decoded transaction detail
`getAccountTransactions(addr)`	Read	Transaction history
`getBlocks(page, size)`	Read	Paginated block list

`getBlockBySlot(slot)`	Read	Single block by slot
`getStats()`	Read	Network overview
`getNetworkEconomics()`	Read	Economics dashboard
`getValidators()`	Read	All validators
`getTokenList()`	Read	All registered tokens
`getContracts()`	Read	All deployed contracts
`getLaunchpads()`	Read	Active bonding curves
`getLaunchpadQuote(id, lamports)`	Read	Preview a launchpad buy
`search(query)`	Read	Universal search
`getSlot()`	Read	Current chain height
`getHealth()`	Read	Node health check

Appendix B: Test Vectors

The following test vectors provide pre-computed byte sequences for common instruction types. Use these to verify that your bincode encoder produces identical output. Each vector shows the raw hex encoding and a field-by-field byte breakdown. If your implementation produces different bytes for the same inputs, your encoding has a bug.

Vector 1: NativeTransfer (5 XRS)

Sends 5 XRS from the address string "Alice" to the address string "Bob". In production, these would be base58-encoded public keys, but short strings are used here for readability. The amount is 5,000,000,000 lamports.

Field	Bytes (hex)	Value
Variant index	`0b000000`	11 (NativeTransfer) as u32 LE
from length	`0500000000000000`	5 as u64 LE
from string	`416c696365`	"Alice" as UTF-8
to length	`0300000000000000`	3 as u64 LE
to string	`426f62`	"Bob" as UTF-8
amount	`00f2052a01000000`	5,000,000,000 as u64 LE

Complete instruction (36 bytes):

```
0b000000050000000000000000416c696365030000000000000000426f6200f2052a01000000
```

Vector 2: Stake (1000 XRS)

Stakes 1,000 XRS (1,000,000,000,000 lamports) for the validator with pubkey string "TestVal".

Field	Bytes (hex)	Value
Variant index	`09000000`	9 (Stake) as u32 LE
pubkey length	`0700000000000000`	7 as u64 LE
pubkey string	`5465737456616c`	"TestVal" as UTF-8
amount	`0010a5d4e8000000`	1,000,000,000,000 as u64 LE

Complete instruction (27 bytes):

```
090000000700000000000000005465737456616c0010a5d4e8000000
```

Vector 3: TokenMint (1 xUSDC)

Mints 1 xUSDC (1,000,000,000 base units at 9 decimals) to the address "Bob". The token_id is "xUSDC".

Field	Bytes (hex)	Value
Variant index	`00000000`	0 (TokenMint) as u32 LE
token_id length	`0500000000000000`	5 as u64 LE
token_id string	`7855534443`	"xUSDC" as UTF-8
to length	`0300000000000000`	3 as u64 LE
to string	`426f62`	"Bob" as UTF-8
amount	`00ca9a3b00000000`	1,000,000,000 as u64 LE

Complete instruction (36 bytes):

```
000000000500000000000000078555344430300000000000000426f6200ca9a3b00000000
```

Vector 4: TokenTransfer (0.5 xUSDC)

Transfers 0.5 xUSDC (500,000,000 base units) from "Alice" to "Bob". The token_id is "xUSDC".

Field	Bytes (hex)	Value
Variant index	`01000000`	1 (TokenTransfer) as u32 LE
token_id length	`0500000000000000`	5 as u64 LE
token_id string	`7855534443`	"xUSDC" as UTF-8
from length	`0500000000000000`	5 as u64 LE
from string	`416c696365`	"Alice" as UTF-8
to length	`0300000000000000`	3 as u64 LE
to string	`426f62`	"Bob" as UTF-8
amount	`0065cd1d00000000`	500,000,000 as u64 LE

Complete instruction (49 bytes):

```
01000000050000000000000078555344430500000000000000416c696365030000000000

0000426f620065cd1d00000000
```

Vector 5: WrapXrs (10 XRS)

Wraps 10 XRS (10,000,000,000 lamports) into the xrs_native token for DEX trading. This is the simplest instruction with only a variant index and an amount field.

Field	Bytes (hex)	Value
Variant index	`0d000000`	13 (WrapXrs) as u32 LE
amount	`00e40b5402000000`	10,000,000,000 as u64 LE

Complete instruction (12 bytes):

```
0d00000000e40b5402000000
```

Encoding Rules Summary

Every instruction begins with a 4-byte u32 in little-endian byte order that identifies the variant. String fields are encoded as a u64 length prefix (8 bytes, little-endian) followed by the raw UTF-8 bytes of the string. There is no null terminator. Integer amounts (u64) are always 8 bytes in little-endian order. Boolean fields are a single byte: 0x00 for false and 0x01 for true. Option fields begin with a 0x00 byte for None or a 0x01 byte followed by the encoded value for Some. Vec<u8> fields use the same encoding as strings: a u64 length prefix followed by the raw bytes.

If your test vector output does not match these hex strings exactly, check three things in order: byte order (must be little-endian), string length prefix size (must be u64 not u32), and variant index (must be u32 not u8). These are the three most common bincode encoding mistakes.

Appendix C: Building DeFi dApps

This appendix covers how to build web-based DeFi applications that run inside the Xeris Command Center wallet browser. When a user opens your dApp in the wallet's built-in browser, the wallet injects a JavaScript provider at window.xeris that your application uses to request transaction signatures. Your code never touches private keys. The wallet handles all signing through a native approval popup that the user must confirm before any transaction is submitted.

How It Works

The Xeris Command Center mobile app includes a dApp browser powered by a WebView. When it loads your web page, it injects a JavaScript provider object (xeris_provider.js) before your page runs. This provider is accessible at window.xeris and also at window.solana for compatibility with tools built for the Solana ecosystem. The provider exposes methods like connect(), signTransaction(), signAndSendTransaction(), and signMessage(). When your dApp calls one of these methods, the provider sends the request to the native Flutter wallet through a JavaScript bridge channel. The wallet shows an approval dialog to the user, signs the transaction with their Ed25519 keypair if approved, and returns the result back to your JavaScript code.

The xeris-sdk npm package includes a class called XerisDApp that wraps this entire flow into simple high-level methods. Instead of manually constructing transactions and calling the provider, you call methods like dapp.transferXrs() or dapp.swapTokens() and the SDK builds the instruction, wraps it in a Solana transaction, sends it to the wallet for signing, and submits it to the network.

XerisClient vs XerisDApp

The SDK provides two main classes for different use cases. XerisClient is for server-side applications, bots, and scripts where your code holds the private key and signs transactions directly. XerisDApp is for browser-based dApps where the user's wallet holds the keys and signs through an approval popup. Both classes share the same instruction encoding and query methods, but they differ in how signing works.

	XerisClient	XerisDApp
Use case	Server, bots, scripts	Browser dApps, DeFi frontends
Keys	You provide a keypair	Wallet manages keys
Signing	SDK signs directly	Wallet popup approves
Environment	Node.js	Browser (WebView)
First argument	XerisKeypair	Not needed (wallet signs)

Installation and Setup

Install the SDK in your web project the same way as any other npm package. If you are building with a bundler like webpack, Vite, or Parcel, the SDK will be bundled into your frontend JavaScript. If you are building a plain HTML page, you can use a CDN or bundle it yourself.

```
npm install xeris-sdk
```

```
const { XerisDApp } = require("xeris-sdk");
```

```
// Or with ES modules:
import { XerisDApp } from "xeris-sdk";
```

Connecting to the Wallet

When your dApp loads, create an XerisDApp instance and call connect(). The SDK automatically detects the injected wallet provider at window.xeris. If the user has not previously approved your dApp, the wallet shows a connection approval popup. Once approved, the connection is remembered for future visits.

```
const dapp = new XerisDApp();
```

```
try {
  await dapp.connect();
  console.log("Connected:", dapp.publicKey);
} catch (err) {
  console.log("User rejected or wallet not found");
}
```

If your script might load before the wallet finishes injecting the provider, use waitForProvider() to poll for it with a timeout.

```
const provider = await XerisDApp.waitForProvider(3000);
if (!provider) {
```

```
  showMessage("Please open this page in the Xeris Command Center");

  return;

}
```

Listening for Events

The dApp adapter emits events when the wallet state changes. Use these to update your UI when the user disconnects or switches accounts.

```
dapp.on("disconnect", () => {

  console.log("Wallet disconnected");

  showLoginScreen();

});




dapp.on("accountChanged", (newPublicKey) => {

  console.log("Switched to:", newPublicKey);

  refreshBalances();

});
```

Sending Transactions

Every transaction method on XerisDApp works the same way internally. It builds the XerisInstruction bytes, wraps them in a Solana transaction with a fresh blockhash, sends the unsigned transaction to the wallet provider, the wallet shows an approval popup, the user taps Approve, the wallet signs and submits the transaction to the node, and the method returns the signature. Your code never sees the private key.

```
// Transfer XRS

const tx = await dapp.transferXrs("recipientAddress", 5.0);

console.log("Signature:", tx.signature);
```

```
// Transfer tokens

await dapp.transferToken("mytoken", "recipientAddress", 100, 9);




// Stake

await dapp.stakeXrs(1000);
```

Swapping Tokens

To swap tokens through an on-chain liquidity pool, first fetch a quote to see the expected output and price impact, then execute the swap with slippage protection. The min_amount_out parameter prevents the transaction from executing if the price moves too far between the quote and the swap.

```
// Get a quote first

const quote = await dapp.getSwapQuote("pool_mtk_xrs", {

  token_in: "mytoken",

  amount_in: "1000000000000"

});

console.log("Expected output:", quote.amount_out);

console.log("Price impact:", quote.price_impact_pct, "%");




// Execute with 5% slippage tolerance

const minOut = Math.floor(quote.amount_out * 0.95);

await dapp.swapTokens("pool_mtk_xrs", "mytoken", 1000000000000, minOut);
```

Buying on a Launchpad

Launchpad bonding curves let users buy new tokens at a price that increases with demand. Always fetch a quote before buying to see the current price and the number of tokens you will receive. The quote also shows the 1% creator fee and 0.777% protocol fee that are deducted from your input.

```
// List available launchpads

const launchpads = await dapp.getLaunchpads();



// Quote: how many tokens for 1 XRS?

const quote = await dapp.getLaunchpadQuote("launch_xyz", 1000000000);

console.log("Tokens out:", quote.tokens_out);



// Buy with 5% slippage tolerance

const minTokens = Math.floor(quote.tokens_out * 0.95);

await dapp.buyOnLaunchpad("launch_xyz", 1000000000, minTokens);



// Sell tokens back to the curve

await dapp.sellOnLaunchpad("launch_xyz", tokenAmount, minXrsOut);
```

Adding and Removing Liquidity

Liquidity providers deposit equal values of two tokens into a pool and receive LP tokens in return. These LP tokens represent their share of the pool and can be redeemed later by calling removeLiquidity.

```
// Add liquidity (must have both tokens)

await dapp.addLiquidity("pool_mtk_xrs", amountTokenA, amountTokenB);



// Remove liquidity (burns LP tokens, returns both tokens)

await dapp.removeLiquidity("pool_mtk_xrs", lpTokenAmount);
```

Wrapping and Unwrapping XRS

The on-chain DEX operates on token balances rather than native XRS balances. Before trading native XRS in a liquidity pool, users must wrap it into the xrs_native token at a 1:1 ratio. After trading, they can unwrap it back to native XRS. Your dApp should handle this automatically when a user wants to trade XRS.

```
// Wrap native XRS into tradeable token

await dapp.wrapXrs(10);




// Trade on the DEX...




// Unwrap back to native XRS

await dapp.unwrapXrs(10);
```

Signing Messages

For authentication or proof of wallet ownership, you can ask the user to sign an arbitrary message. This does not create a transaction or cost any fees. The wallet shows the message content in the approval popup so the user knows what they are signing.

```
const message = "Sign in to MyDApp at " + new Date().toISOString();

const { signature } = await dapp.signMessage(message);

// Send the signature to your backend to verify ownership
```

Reading Chain Data

All read-only queries work without wallet approval. They use the RPC URL provided by the wallet (or the default testnet node). Use these to display balances, transaction history, pool states, and network statistics in your dApp UI.

```
// Connected wallet balance

const balance = await dapp.getBalance();

const xrs = balance / 1_000_000_000;
```

```
// All token balances

const accounts = await dapp.getTokenAccounts();




// Pool state

const pool = await dapp.getContract("pool_mtk_xrs");




// Network stats

const stats = await dapp.getStats();




// Transaction history

const tx = await dapp.getTransaction(signatureStr);
```

Complete DeFi dApp Example

The following example shows a minimal DeFi frontend that connects to the wallet, displays the user's balance, and lets them swap tokens. This pattern works in any web framework including React, Vue, Svelte, or plain HTML.

```
const { XerisDApp } = require("xeris-sdk");




const dapp = new XerisDApp();




async function init() {

  await dapp.connect();

  document.getElementById("address").textContent = dapp.publicKey;
```

```
  const bal = await dapp.getBalance();
  document.getElementById("balance").textContent =
    (bal / 1_000_000_000).toFixed(2) + " XRS";


  const tokens = await dapp.getTokenAccounts();
  tokens.token_accounts.forEach(t => {
    addTokenRow(t.symbol, t.balance_display);
  });
}


async function swap(poolId, tokenIn, amountIn) {
  const quote = await dapp.getSwapQuote(poolId, {
    token_in: tokenIn,
    amount_in: String(amountIn)
  });
  const minOut = Math.floor(quote.amount_out * 0.95);
  const tx = await dapp.swapTokens(poolId, tokenIn, amountIn, minOut);
  showSuccess("Swap confirmed: " + tx.signature);
}


init().catch(err => showError(err.message));
```

XerisDApp Method Reference

The following table lists every method available on XerisDApp. Write methods trigger the wallet's approval popup. Read methods execute without approval.

Method	Type	Description
`connect(opts?)`	Write	Connect to the wallet
`disconnect()`	Write	Disconnect from wallet
`transferXrs(to, xrs)`	Write	Transfer native XRS
`stakeXrs(xrs)`	Write	Stake for mining
`unstakeXrs(xrs)`	Write	Begin unstaking
`transferToken(id, to, amt, d)`	Write	Transfer tokens
`swapTokens(pool, tokenIn, amtIn, min)`	Write	Swap on a DEX pool
`buyOnLaunchpad(id, xrs, minOut)`	Write	Buy on bonding curve
`sellOnLaunchpad(id, tokens, minXrs)`	Write	Sell on bonding curve
`addLiquidity(pool, amtA, amtB)`	Write	Provide liquidity
`removeLiquidity(pool, lp)`	Write	Remove liquidity
`wrapXrs(xrs)`	Write	Wrap XRS for DEX
`unwrapXrs(xrs)`	Write	Unwrap to native XRS
`callContract(id, method, args)`	Write	Call any contract
`signMessage(msg)`	Write	Sign for authentication
`sendInstruction(ixData)`	Write	Send raw instruction
`getBalance(addr?)`	Read	XRS balance in lamports
`getTokenAccounts(addr?)`	Read	All token balances
`getAccountInfo(addr?)`	Read	Full account info
`getLaunchpads()`	Read	Active bonding curves
`getLaunchpadQuote(id, lamports)`	Read	Preview launchpad buy
`getContracts()`	Read	All deployed contracts
`getContract(id)`	Read	Single contract state
`getSwapQuote(id, params)`	Read	Preview a swap
`getTokenList()`	Read	All registered tokens
`getStats()`	Read	Network overview
`getTransaction(sig)`	Read	Transaction detail
`airdrop(xrs)`	Read	Testnet faucet

Wallet Provider API (Advanced)

If you need lower-level access than XerisDApp provides, you can interact with the injected wallet provider directly. The provider is available at window.xeris and implements the following methods: connect(), disconnect(), signTransaction(), signAllTransactions(), signAndSendTransaction(), signMessage(), and getRpcUrl(). Each method returns a Promise that resolves after the user approves or rejects the request in the wallet popup.

```
const provider = window.xeris || window.solana;

if (!provider) throw new Error("No wallet found");




const { publicKey } = await provider.connect();

console.log("Connected:", publicKey.toString());




// Sign and send a raw transaction

const { signature } = await provider.signAndSendTransaction(solanaTransaction);




// Listen for account changes

provider.on("accountChanged", (newKey) => {

  console.log("Account changed:", newKey?.toString());

});
```

The provider is injected by the Xeris Command Center's dApp browser (xeris_provider.js). It communicates with the native Flutter wallet through a JavaScript bridge channel called XerisWallet. The native side handles key management, transaction signing, and RPC submission. The complete provider source and the Flutter bridge (xeris_wallet_bridge.dart) are available in the Xeris GitHub repository for wallet developers who want to integrate the same protocol into their own applications.

XerisCoin Testnet SDK Reference v1.4

Xeris Technologies LLC
https://xerisweb.com
For questions or issues, contact the Xeris team or Zachary Winkle directly at zachary@xerisweb.com
https://t.me/xeriscoin
https://x.com/xeriscoin